

제07장.

- 프로젝트 생성을 위한 `express-generator` 모듈을 전역으로 설치

```
npm install express-generator -g
```

- 프로젝트 실행을 위한 `nodemon` 모듈을 전역으로 설치

```
npm install nodemon -g
```

- `express-generator` 모듈을 이용한 프로젝트 생성

```
C:\data>express -e web
```

- 생성된 프로젝트 폴더에서 `package.json`에 선언된 모듈 설치

```
C:\data\web>npm install
```

- 부트스트랩 적용

1) 구글 폰트 변경

```
[public]-[styleSheets] style.css
```

```
@import url('https://fonts.googleapis.com/css2?family=Gowun+Batang&display=swap');
```

```
body {  
  font-family: "Gowun Batang", serif;  
  font-weight: 400;  
  font-style: normal;  
}
```

2) 부트스트랩의 네비게이션 바를 복사하여 메뉴 바를 작성한다.

```
[views] menubar.ejs
```

```
<nav class="navbar navbar-expand-lg bg-primary data-bs-theme="dark">  
  <div class="container-fluid">  
    <a class="navbar-brand" href="/">HOME</a>  
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent"  
      aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle navigation">  
      <span class="navbar-toggler-icon"></span>  
    </button>  
    <div class="collapse navbar-collapse" id="navbarSupportedContent">  
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">  
        <li class="nav-item">  
          <a class="nav-link active" aria-current="page" href="#">게시글</a>  
        </li>  
      </ul>  
      <ul class="navbar-nav">  
        <li class="navbar-nav me-auto mb-2 mb-lg-0">  
          <a class="nav-link active" aria-current="page" href="/users/login">로그인</a>  
        </li>  
      </ul>  
    </div>  
  </div>  
</nav>
```

3) 부트스트랩 공식 문서 사이트에서 CDN 주소를 복사하여 index.ejs 파일 head 태그에 붙여 넣는다.

```
[views] index.ejs
```

```
<!DOCTYPE html>
<html>
<head>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
    rel="stylesheet"
    integrity="sha384-sRII4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB"
    crossorigin="anonymous">
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKyoEForCGlyvwx9Hj09IcYn3nv7wiPVlz7YYwlrWVcXK/BmnVDxM+D2scQbITxI"
    crossorigin="anonymous"></script>
  <title><%=title%></title>
  <link rel='stylesheet' href='/stylesheets/style.css'/>
</head>
<body>
  <div class="container">
    <%-include("menubar")%>
  </div>
</body>
</html>
```

• HomePage 작성

홈페이지를 작성하여 index 라우터에 pageName 변수에 저장한 후 index 페이지에서 pageName을 Include한다.

```
[views] home.ejs
```

```
<div>
  <h1 class="text-center my-5"><%=title%></h1>
  <h3 class="text-center">홈페이지에 오신걸 환영합니다.</h3>
</div>
```

```
[routes] index.js
```

```
var express = require('express');
var router = express.Router();

/* GET home page. */
router.get('/', async function (req, res, next) {
  res.render('index', { title: '홈페이지', pageName:'home.ejs' });
});

module.exports = router;
```

```
[views] index.ejs
```

```
<html>
<head>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet" ...>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"...></script>
  <title><%=title%></title>
  <link rel='stylesheet' href='/stylesheets/style.css'/>
</head>
<body>
  <div class="container">
    <%-include("menubar")%>
    <%-include(pageName)%>
  </div>
</body>
</html>
```

• 로그인 페이지 작성

1) 로그인 페이지를 작성하여 users 라우터 pageName 변수에 저장한다.

```
[routes] users.js
```

```
router.get('/login', function (req, res, next) {  
  res.render('index', { title: '로그인' , pageName:'users/login.ejs'});  
});
```

```
[views]-[users] login.ejs
```

```
<div class="row justify-content-center my-5">  
  <div class="col col-xl-3 col-lg-4 col-md-5 col-sm-6">  
    <div class="card">  
      <div class="card-header text-center">  
        <h3>로그인</h3>  
      </div>  
      <div class="card-body">  
        <form name="frm" type="post">  
          <input name="id" class="form-control mb-2"/>  
          <input name="pass" type="password" class="form-control mb-2"/>  
          <button type="submit" class="btn btn-primary w-100">로그인</button>  
        </form>  
      </div>  
    </div>  
  </div>  
</div>
```

2) Oracle Developer 에서 새로운 유저 생성과 권한을 부여한다.

```
create user board identified by pass;  
grant connect, resource, dba to board;
```

3) 사용자 관리 테이블을 생성하고 샘플 데이터를 등록한다.

```
CREATE TABLE users(  
  id VARCHAR2(20) PRIMARY KEY, pass VARCHAR2(100), name VARCHAR2(20), join_date TIMESTAMP  
);  
INSERT INTO users(id, pass, name, join_date) VALUES('blue', 'pass', '김블루', SYSDATE);  
INSERT INTO users(id, pass, name, join_date) VALUES('pink', 'pass', '이핑크', SYSDATE);
```

4) 아이디와 비밀번호를 입력 받아 로그인 체크 함수를 작성한다.

```
CREATE OR REPLACE FUNCTION LOGIN_CHECK(i_id VARCHAR2, i_pass VARCHAR2)  
RETURN INT AS  
  v_result INT := 0; --아이디가 없는 경우  
  v_count INT;  
BEGIN  
  FOR rec IN (SELECT * FROM users WHERE id=i_id) LOOP  
    IF rec.pass = i_pass THEN  
      v_result := 1; --로그인 성공한 경우  
    ELSE  
      v_result := 2; --비밀번호가 불일치한 경우  
    END IF;  
  END LOOP;  
  RETURN v_result;  
END LOGIN_CHECK;
```

```
SELECT LOGIN_CHECK('blue', 'pass') AS result FROM dual;
```

5) 오라클 데이터베이스 접속을 위한 라이브러리를 설치

```
npm install oracledb
```

6) 오라클 데이터베이스 연결 함수를 작성한다.

connect.js

```
const oracledb = require('oracledb');

//1. Oracle버전 12미만인 경우 Thick 모드 활성화 (반드시 최상단에서 실행)
try {
  oracledb.initOracleClient({ libDir:'C:\\data\\oracle\\instantclient_19_30' });
} catch(err){
  console.log('오라클 클라이언트 초기화 실패', err);
  process.exit(1);
}

//2. 연결 함수 내보기기 (외부에서 사용 가능)
async function getConnection() {
  let connection;
  try{
    connection = await oracledb.getConnection({
      user:'board',
      password:'pass',
      connectionString:'localhost:1521/xe'
    });
    console.log('Oracle DB 연결 성공!');
    return connection;
  } catch(err){
    console.log('Connect 오류 발생:', err);
  }
}

module.exports = { getConnection };
```

7) 로그인 체크 결과를 위한 라우터를 추가한다.

[routes] users.js

```
var express = require('express');
var router = express.Router();
const { getConnection } = require('../connect');
const oracledb = require('oracledb');

router.post('/login', async function(req, res){
  let id=req.body.id;
  let pass=req.body.pass;
  console.log(id, pass);
  let con;
  try{
    con = await getConnection();
    const sql ='select login_check(:id, :pass) as login from dual';
    const result = await con.execute(sql, [id, pass], {outFormat:oracledb.OUT_FORMAT_OBJECT});
    //console.log(result.rows[0].LOGIN);
    res.send(result.rows[0].LOGIN.toString());
  } catch(err){
    console.log('에러:', err.message);
  } finally{
    if(con) await con.close();
  }
});

module.exports = router;
```

8) 로그인 체크를 위한 웹페이지를 완성한다.

[views] index.ejs

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js"></script>
```

[views]-[users] login.ejs

```
<script>
  $(frm).on("submit", function(e){
    e.preventDefault();
    const id = $(frm.id).val();
    const pass = $(frm.pass).val();
    $.ajax({
      type:'post',
      url:'/users/login',
      data: {id, pass},
      success: function(data){
        if(data==1){
          localStorage.setItem('id', id);
          location.href='/';
        } else if(data==2) {
          alert('비밀번호가 일치하지 않습니다!');
        } else if(data==0){
          alert('아이디가 존재하지 않습니다.');
        }
      }
    });
  });
</script>
```

[views] menubar.ejs

```
<nav class="navbar navbar-expand-lg bg-primary" data-bs-theme="dark">
  <div class="container-fluid">
    ...
    <ul id="before" class="navbar-nav">
      <li class="navbar-nav me-auto mb-2 mb-lg-0">
        <a class="nav-link active" aria-current="page" href="/users/login">로그인</a>
      </li>
    </ul>
    <ul id="after" class="navbar-nav">
      <li class="navbar-nav me-auto mb-2 mb-lg-0">
        <a id="id" class="nav-link active" aria-current="page" href="#">아이디</a>
      </li>
      <li class="navbar-nav me-auto mb-2 mb-lg-0">
        <a id="logout" class="nav-link active" aria-current="page" href="#">로그아웃</a>
      </li>
    </ul>
  </div>
</div>
</nav>
<script>
  const id = localStorage.getItem("id");
  if(id){
    $("#before").hide(); $("#after").show();
    $("#id").html(`${id}님`);
  } else{
    $("#before").show(); $("#after").hide();
  }

  $("#logout").on("click", function(){
    if(confirm("정말로 로그아웃 하실래요?")){
      localStorage.clear();
      location.href="/";
    }
  });
</script>
```

• 게시글 목록

1) 게시글 관리를 위한 테이블을 생성하고 샘플 데이터를 입력한다.

```
CREATE SEQUENCE SEQ_POST_ID START WITH 1 INCREMENT BY 1;
```

```
CREATE TABLE posts(  
    id INT PRIMARY KEY,  
    title VARCHAR2(1000),  
    body VARCHAR2(2000),  
    writer VARCHAR2(20),  
    reg_date TIMESTAMP DEFAULT SYSDATE,  
    FOREIGN KEY(writer) REFERENCES users(id)  
);
```

```
INSERT INTO posts(id, title, body, writer)  
VALUES(SEQ_POST_ID.NEXTVAL, '절차적 기능','SQL의 단점인 순차적 처리 능력을 해결하여 IF, FOR, WHILE 등의 문법을 제공한다.','blue');  
INSERT INTO posts(id, title, body, writer)  
VALUES(SEQ_POST_ID.NEXTVAL, '성능 향상','다수의 SQL 문을 한 번에 블록 단위로 DB 서버에 전달하여 네트워크 트래픽을 감소시킨다.','pink');  
INSERT INTO posts(id, title, body, writer)  
INSERT INTO posts(id, title, body, writer)  
VALUES(SEQ_POST_ID.NEXTVAL, '예외 처리 (Exception Handling)','런타임 에러 발생 시 프로그램이 멈추지 않고 적절하게 대응하도록 처리한다.','blue');  
INSERT INTO posts(id, title, body, writer)  
INSERT INTO posts(id, title, body, writer)  
VALUES(SEQ_POST_ID.NEXTVAL, '프로시저란?','프로시저는 SQL을 확장하여 고성능의 트랜잭션 처리 언어이다.','pink');  
INSERT INTO posts(id, title, body, writer)  
VALUES(SEQ_POST_ID.NEXTVAL, '프로시저의 장점','프로시저는 모든 SQL 데이터의 조작, 커서제어, 트랜잭션 제어, SQL 함수 등을 사용할 수 있다.','blue');
```

2) 검색과 페이징 처리를 위한 함수를 작성한다.

```
CREATE OR REPLACE PROCEDURE SEARCH_POSTS(  
    i_word IN VARCHAR2,  
    i_page IN INT,  
    i_size IN INT,  
    o_cur OUT SYS_REFCURSOR,  
    o_count OUT INT)  
AS  
    v_sql VARCHAR2(1000);  
    v_tbl VARCHAR2(1000);  
    v_start INT := (i_page-1) * i_size + 1;  
    v_end INT := (i_page * i_size);  
    v_word VARCHAR2(100) := '%' || i_word || '%';  
BEGIN  
    v_tbl := '(SELECT posts.*, ROW_NUMBER() OVER(ORDER BY id) AS rn FROM posts WHERE title LIKE :1 OR body LIKE :2)';  
    v_sql := 'SELECT * FROM ';  
    v_sql := v_sql || v_tbl || ' WHERE rn BETWEEN :3 AND :4';  
    OPEN o_cur FOR v_sql USING v_word, v_word, v_start, v_end;  
    v_sql := 'SELECT COUNT(*) FROM posts WHERE title LIKE :1 OR body LIKE :2';  
    EXECUTE IMMEDIATE v_sql INTO o_count USING v_word, v_word;  
END SEARCH_POSTS;
```

```
var cur refcursor;  
var cnt number;  
exec SEARCH_POSTS('',2, 3, :cur, :cnt);  
print cur;  
print cnt;
```

3) 게시물 관련 라우터를 생성하여 app.js 파일에 등록한다.

[routes] posts.js

```
var express = require('express');
var router = express.Router();
var { getConnection } = require('../connect');
var oracledb = require('oracledb');
const moment = require("moment");

/* 게시물 목록 페이지 이동 */
router.get('/', async function (req, res, next) {
  res.render('index', { title: '게시글', pageName:'posts/list.ejs' });
});

/* 게시물 목록 데이터 생성 */
router.get('/list.json', async function(req, res){
  const word = req.query.word || '';
  const page = parseInt(req.query.page) || 1;
  const size = parseInt(req.query.size) || 5;
  //console.log(page, word, size);
  let con;
  try{
    con = await getConnection();
    const sql = `BEGIN SEARCH_POSTS(:word, :page, :size, :o_cur, :o_cnt); END;`;
    const binds = {
      word,
      page,
      size,
      o_cur: { dir:oracledb.BIND_OUT, type:oracledb.CURSOR },
      o_cnt: { dir:oracledb.BIND_OUT, type:oracledb.NUMBER}
    }
    const result = await con.execute(sql, binds, { outFormat:oracledb.OUT_FORMAT_OBJECT });
    // console.log('게시글 수', result.outBinds.o_cnt);
    const resultSet = result.outBinds.o_cur;
    const rows = [];
    let row;
    while(row= await resultSet.getRow()){
      rows.push({
        ...row,
        DATE:moment(row.REG_DATE).format('YYYY-MM-DD HH:mm:ss')
      });
    }
    //console.log(rows);
    res.send({ count: result.outBinds.o_cnt, posts:rows });
  }catch(err){
    console.log('오류:', err.message);
  }finally{
    if(con) await con.close();
  }
});

module.exports = router;
```

app.js

```
var app = express();

app.use('/', indexRouter);
app.use('/users', usersRouter);
app.use('/posts', require('./routes/posts'));

module.exports = app;
```

4) 게시물 목록 페이지를 작성한다.

[views]-[posts] list.ejs

```
<h1 class="text-center my-5">게시글</h1>
<div class="text-end mb-2">
  <button class="btn btn-primary px-5">새글작성</button>
</div>
<table id="tbl" class="table table-striped table-hover"></table>
<div class="text-center">
  <button id="prev" class="btn btn-primary">이전</button>
  <span id="page" class="px-3">1/10</span>
  <button id="next" class="btn btn-primary">다음</button>
</div>
<script id="temp" type="text/x-handlebars-template">
  <tbody>
    {{#each posts}}
      <tr>
        <td>{{ID}}</td>
        <td>{{TITLE}}</td>
        <td>{{WRITER}}</td>
        <td class="text-end">{{DATE}}</td>
      </tr>
    {{/each}}
  </tbody>
</script>
<script>
let page=1;
let size=5;
let word="";
//목록 출력 함수실행
getList();
//페이지 이동 이전, 다음버튼을 클릭한 경우
$("#next").on("click", function(){
  page++;
  getList();
});
$("#prev").on("click", function(){
  page--;
  getList();
});
//목록 출력 함수
function getList(){
  $.ajax({
    type:"get",
    url:"/posts/list.json",
    data: { page, size, word },
    success:function(data){
      console.log(data);
      const temp = Handlebars.compile($("#temp").html());
      $("#tbl").html(temp(data));
      //페이지 정보를 출력한다.
      const last = Math.ceil(data.count/size);
      $("#page").html(`${page}/${last}`);
      if(page==1) $("#prev").attr("disabled", true);
      else $("#prev").attr("disabled", false);
      if(page==last) $("#next").attr("disabled", true);
      else $("#next").attr("disabled", false);
    }
  })
}
</script>
```

- 게시글 등록

[routes] posts.js

```
router.get('/insert', function(req, res){
  res.render('index', { title:'게시글 등록', pageName:'posts/insert.ejs'});
});

router.post('/insert', async function(req, res){
  const title=req.body.title;
  const body=req.body.body;
  const writer=req.body.writer;
  let con;
  try{
    con = await getConnection();
    const sql='insert into posts(id, title, body, writer) values(seq_post_id.nextval, :title, :body, :writer)';
    await con.execute(sql, { title, body, writer }, { autoCommit:true });
    res.sendStatus(200);
  }catch(err){
    console.log('오류:', err.message);
  }finally{
    if(con) await con.close();
  }
});
```

[views]-[posts] insert.ejs

```
<h1 class="text-center my-5">글쓰기</h1>
<div class="row justify-content-center">
  <div class="col-lg-8">
    <form name="frm" method="post">
      <input name="title" placeholder="제목을 입력 하세요." class="form-control mb-2"/>
      <textarea name="body" class="form-control" rows="10" placeholder="내용을 입력 하세요."></textarea>
      <div class="text-center mt-2">
        <button type="submit" class="btn btn-primary px-5">등록</button>
        <button type="reset" class="btn btn-secondary px-5">취소</button>
      </div>
    </form>
  </div>
</div>
<script>
  $(frm).on("submit", function(e){
    e.preventDefault();
    const title=$(frm.title).val();
    const body=$(frm.body).val();
    if(title==""){
      alert("제목을 입력하세요!");
      $(frm.title).focus();
    }else if(body==""){
      alert("내용을 입력하세요!");
      $(frm.body).focus();
    }else{
      $.ajax({
        type:"post",
        url:"/posts/insert",
        data:{ title, body, writer:id },
        success:function(data){
          location.href="/posts";
        }
      });
    }
  });
</script>
```

- 게시물 정보 삭제

1) 게시물 정보 출력, 삭제 라우터를 작성한다.

[routes] posts.js

```
/* 게시물 정보 페이지 이동 */
router.get('/:id', async function(req, res){
  const id = req.params.id;
  let con;
  try{
    con = await getConnection();
    const sql = `select * from posts where id=:id`;
    const result = await con.execute(sql, {id}, {outFormat:oracledb.OUT_FORMAT_OBJECT});
    const post = { ...result.rows[0], DATE:moment(result.rows[0].REG_DATE).format('YYYY-MM-DD HH:mm:ss') };
    //console.log(post);
    res.render('index', { title:'게시글 정보', post, pageName:'posts/read.ejs' });
  }catch(err){
    console.log('오류:', err.message);
  }finally{
    if(con) await con.close();
  }
});

/*게시글 삭제 */
router.post('/delete', async function(req, res){
  const id = req.body.id;
  let con;
  try{
    con = await getConnection();
    const sql = `delete from posts where id=:id`;
    await con.execute(sql, { id }, { autoCommit:true });
    res.sendStatus(200);
  }catch(err){
    console.log('오류:', err.message);
  }finally{
    if(con) await con.close();
  }
});
```

2) 게시물 정보를 출력하는 페이지를 작성한다.

[views]-[posts] read.ejs

```
<h1 class="text-center my-5">게시글 정보</h1>
<div class="row justify-content-center">
  <div class="col-lg-8">
    <div id="buttons" class="text-end mb-2">
      <button class="btn btn-outline-primary">수정</button>
      <button class="btn btn-outline-danger">삭제</button>
    </div>
    <div class="card">
      <div class="card-header">
        <%=post.ID%>.<%=post.TITLE%>
      </div>
      <div class="card-body" style="white-space:pre-wrap;"><%=post.BODY%></div>
      <div class="card-footer">
        Create on <%=post.DATE%> by <%=post.WRITER%>
      </div>
    </div>
  </div>
</div>
```

```
<script>
  const writer = "<%=post.WRITER%>";
  const postId = "<%=post.ID%>";
  //로그인한 아이디와 글쓴이가 일치하는지 체크
  if(id == writer){
    $("#buttons").show();
  }else{
    $("#buttons").hide();
  }
  //삭제 버튼을 클릭한 경우
  $(".btn-outline-danger").on("click", function(){
    if(confirm(`${postId}번 게시글을 삭제하실래요?`)){
      $.ajax({
        type:"post",
        url:"/posts/delete",
        data: { id: postId },
        success: function(data){
          if(data=='OK'){
            alert("삭제가 완료되었습니다.");
            location.href="/posts";
          }
        }
      });
    }
  });
  //수정 버튼을 클릭한 경우
  $(".btn-outline-primary").on("click", function(){
    location.href="/posts/update/${postId}";
  });
</script>
```

• 게시글 수정

1) 수정 페이지 이동 라우터와 수정페이지를 작성한다.

```
/* 게시글 정보 페이지 이동 */
router.get('/:id', async function(req, res){
  ...
});

/*게시글 수정 페이지 이동*/
router.get('/update/:id', async function(req, res){
  const id = req.params.id;
  let con;
  try{
    con = await getConnection();
    const sql = `select * from posts where id=:id`;
    const result = await con.execute(sql, { id }, { outFormat:oracledb.OUT_FORMAT_OBJECT });
    //console.log(result.rows[0]);
    const post = { ...result.rows[0] };
    res.render('index', { title:'게시글 수정', post, pageName:'posts/update.ejs' });
  }catch(err){
    console.log('오류:', err.message);
  }finally{
    if(con) await con.close();
  }
});
```

[views]-[posts] update.ejs

```
<h1 class="text-center my-5">게시글 수정</h1>
<div class="row justify-content-center">
  <div class="col-lg-8">
    <form name="frm" method="post">
      <input name="title" value="<%=post.TITLE%>" placeholder="제목을 입력 하세요" class="form-control mb-2"/>
      <textarea name="body" class="form-control" rows="10" placeholder="내용을 입력 하세요"><%=post.BODY%></textarea>
      <div class="text-center mt-2">
        <button type="submit" class="btn btn-primary px-5">등록</button>
        <button type="reset" class="btn btn-secondary px-5">취소</button>
      </div>
    </form>
  </div>
</div>
<script>
  const postId = "<%=post.ID%>";
  //등록 버튼을 클릭한 경우
  $(frm).on("submit", function(e){
    const title = $(frm.title).val();
    const body = $(frm.body).val();
    //console.log(title, body, postId);
    e.preventDefault();
    if(confirm(`${postId}번 게시글을 수정하시겠습니까?`)){
      $.ajax({
        type:"post",
        url:"/posts/update",
        data: { id:postId, title, body },
        success:function(data){
          if(data=='OK') {
            alert("수정이 완료되었습니다.");
            location.href=`/posts/${postId}`;
          }
        }
      });
    }
  });
</script>
```

2) 게시글 정보를 받아 수정하는 라우터를 작성한다.

[routes] posts.js

```
/*게시글 수정*/
router.post('/update', async function(req, res){
  const id = req.body.id;
  const title = req.body.title;
  const body = req.body.body;
  let con;
  try{
    con = await getConnection();
    sql =`update posts set title=:title, body=:body where id=:id`;
    await con.execute(sql, { title, body, id }, { autoCommit:true });
    res.sendStatus(200);
  }catch(err){
    console.log('오류', err.message);
  }finally{
    if(con) await con.close();
  }
});
module.exports = router;
```